

Оглавление

Практическая работа №1	2
Практическая работа №2	9
Практическая работа №3	14
Практическая работа №4	19
Практическая работа №5	25
Практическая работа №6	38

Практическая работа №1.

Структура программы на языке Си++. Процедуры ввода-вывода. Операторы присвоения.

1. Структура программы на C++

Любая программа на C++ состоит из набора функций. Самая главная из них — `main()`. Выполнение программы начинается именно с нее.

Базовая структура:

```
// 1. Директивы препроцессора (подключение библиотек)
#include <iostream>
// 2. Пространства имен (чтобы не писать std:: перед каждой командой)
using namespace std;
// 3. Главная функция программы
int main() {
// 4. Тело функции - здесь пишется основная логика программы
// 5. Возврат значения операционной системе (0 - обычно означает успешное завершение)
return 0;
}
```

Разберем каждую часть:

- **#include <iostream>** — это директива препроцессора. Она "подключает" библиотеку `iostream`, которая необходима для операций ввода и вывода (таких как `cout` и `cin`).
- **using namespace std;** — объявляет, что мы используем стандартное пространство имен `std`. Это позволяет вместо `std::cout` писать просто `cout`. Без этой строки пришлось бы постоянно указывать пространство имен.
- **int main()** — объявление главной функции. Тип `int` означает, что функция должна вернуть целое число.
- **{ ... }** — фигурные скобки обозначают начало и конец тела функции.
- **return 0;** — оператор возврата. Значение 0 сигнализирует операционной системе, что программа завершилась успешно.

2. Процедуры ввода-вывода

Для ввода и вывода данных в C++ используются потоки `cout` (вывод) и `cin` (ввод). Они определены в библиотеке `<iostream>`.

Вывод данных (cout)

Оператор `cout` (console output) используется для вывода данных на экран.

Синтаксис: `cout << данные;`

Примеры:

```
#include <iostream>
using namespace std;

int main() {
    // Вывод строки
    cout << "Hello, World!";

    // Вывод числа
    int age = 25;
    cout << age;

    // Вывод нескольких значений подряд
    string name = "Alice";
    cout << "Name: " << name << ", Age: " << age << endl; // endl - переводит на новую строку

    // Вывод с переходом на новую строку (два способа)
```

```

cout << "This is first line." << endl;
cout << "This is second line.\n"; // Символ '\n' также переводит курсор
cout << "This is third line.";

return 0;
}

```

Ввод данных (cin)

Оператор **cin** (console input) используется для чтения данных, введенных пользователем с клавиатуры.

Синтаксис: `cin >> переменная;`

Примеры:

```

#include <iostream>
using namespace std;

int main() {
    int number;
    string name;
    double price;

    // Ввод одного числа
    cout << "Enter a number: ";
    cin >> number;

    // Ввод строки (до первого пробела)
    cout << "Enter your name: ";
    cin >> name;

    // Ввод нескольких значений подряд
    cout << "Enter your name, age and salary: ";
    cin >> name >> number >> price; // Пользователь вводит, например: Bob 30 50000.5

    // Вывод для проверки
    cout << "Hi " << name << ", you are " << number << " years old and earn $" << price << endl;

    return 0;
}

```

Важно: Если нужно считать строку с пробелами, используйте **getline(cin, variable)**.

```

#include <iostream>
#include <string> // Необходима для работы с string
using namespace std;

int main() {
    string fullName;
    cout << "Enter your full name: ";
    getline(cin, fullName); // Считает всю строку, включая пробелы
    cout << "Hello, " << fullName << "!";
    return 0;
}

```

3. Операторы присваивания

Оператор присваивания = записывает значение в переменную.

Базовый синтаксис: `переменная = выражение;`

Примеры:

```

#include <iostream>
using namespace std;

int main() {
    // Простое присваивание
    int a;
    a = 10; // Переменной a присваивается значение 10

    // Присваивание при объявлении (инициализация)
}

```

```

int b = 20;
double pi = 3.14159;
char letter = 'A';
string greeting = "Hello";

// Присваивание результата выражения
int x = 5;
int y = 3;
int z = x + y; // z будет равно 8

// Множественное присваивание
int p, q, r;
p = q = r = 100; // Всем трем переменным присваивается значение 100

// Составные операторы присваивания (сокращенная запись)
int counter = 5;
counter += 3; // То же, что counter = counter + 3; (теперь counter = 8)
counter -= 2; // То же, что counter = counter - 2; (теперь counter = 6)
counter *= 4; // То же, что counter = counter * 4; (теперь counter = 24)
counter /= 3; // То же, что counter = counter / 3; (теперь counter = 8)

// Также существуют %=, &=, |=, <<=, >>= для других операций

return 0;
}

```

Пример законченной программы:

```

#include <iostream>
using namespace std;

int main() {
    // Объявление переменных
    int num1, num2, sum;

    // Ввод данных от пользователя
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;

    // Присваивание результата выражения
    sum = num1 + num2;

    // Вывод результата
    cout << "The sum of " << num1 << " and " << num2 << " is " << sum << endl;

    return 0;
}

```

4. Типы данных в C++

Тип	Название	Диапазон значений	Размер
Целочисленные со знаком			
char	символьный	-128 до 127	1 байт
short	короткий целый	-32,768 до 32,767	2 байта
int	целый	-2 ¹⁵ до 2 ¹⁵ -1 (-2,147,483,648 до 2,147,483,647)	4 байта
long	длинный целый	-2 ³¹ до 2 ³¹ -1	4 байта
long long	очень длинный целый	-2 ⁶³ до 2 ⁶³ -1	8 байт
Целочисленные без знака			
unsigned char	беззнаковый символьный	0 до 255	1 байт
unsigned short	беззнаковый короткий	0 до 65,535	2 байта
unsigned int	беззнаковый целый	0 до 4,294,967,295	4 байта
unsigned long	беззнаковый длинный	0 до 4,294,967,295	4 байта
unsigned long long	беззнаковый очень длинный	0 до 18,446,744,073,709,551,615	8 байт
Вещественные (с плавающей точкой)			
float	вещественный одинарной точности	±1.2×10 ⁻³⁸ до ±3.4×10 ³⁸ (~7 значащих цифр)	4 байта
double	вещественный двойной точности	±2.2×10 ⁻³⁰⁸ до ±1.8×10 ³⁰⁸ (~15 значащих цифр)	8 байт

long double	вещественный повышенной точности	$\pm 3.4 \times 10^{-4932}$ до $\pm 1.2 \times 10^{4932}$ (~19 значащих цифр)	10-16 байт
Логический			
bool	логический	true (1) или false (0)	1 байт
Строковый			
string	строковый	Произвольная длина (ограничена памятью)	Зависит от длины строки
Специальные типы размеров			
size_t	для хранения размеров	Зависит от платформы (обычно 4 или 8 байт)	
int8_t, int16_t, int32_t, int64_t	типы фиксированного размера	1, 2, 4, 8 байт соответственно	

Примеры:

<i>Целочисленные типы</i>	<i>Типы с плавающей точкой</i>
<i>// Со знаком (могут хранить отрицательные значения)</i> short s = 100; // 2 байта, обычно от -32768 до 32767 int i = 100000; // 4 байта, обычно от -2^{15} до $2^{15}-1$ long l = 1000000L; // 4 или 8 байт long long ll = 1000000000LL; // 8 байт <i>// Без знака (только неотрицательные значения)</i> unsigned short us = 200; unsigned int ui = 400000; unsigned long ul = 4000000UL; <i>// Символьный тип</i> char ch = 'A'; // 1 байт, хранит ASCII-код символа unsigned char uch = 200;	float f = 3.14f; // 4 байта, ~7 значащих цифр double d = 3.141592; // 8 байт, ~15 значащих цифр long double ld = 3.14159265358979L; // 10+ байт
<i>Логический тип</i>	<i>Строковый тип</i>
bool b1 = true; // истина (1) bool b2 = false; // ложь (0)	#include <string> using namespace std; string str1 = "Hello"; string str2 = "World";

Определение размеров типов

```
#include <iostream>
using namespace std;
```

```
int main() {
    cout << "Size of int: " << sizeof(int) << " bytes" << endl;
    cout << "Size of double: " << sizeof(double) << " bytes" << endl;
    cout << "Size of char: " << sizeof(char) << " byte" << endl;
    cout << "Size of bool: " << sizeof(bool) << " byte" << endl;
    return 0;
}
```

5. Действия с целыми числами

Арифметические операции

Операция	Оператор	Пример	Результат	Примечания
<i>Сложение</i>	+	int a = 5 + 3;	8	Сумма двух операндов
<i>Вычитание</i>	-	int b = 10 - 4;	6	Разность двух операндов
<i>Умножение</i>	*	int c = 6 * 7;	42	Произведение двух операндов
<i>Деление</i>	/	int d = 15 / 4;	3	Целочисленное деление (для целых типов)
	/	double e = 15.0 / 4.0;	3.75	Обычное деление (для вещественных типов)
<i>Остаток от деления</i>	%	int f = 17 % 5;	2	Только для целых типов
<i>Инкремент (префиксный)</i>	++	int x = 5; int y = ++x;	x=6, y=6	Сначала увеличивает, потом использует
<i>Инкремент (постфиксный)</i>	++	int x = 5; int y = x++;	x=6, y=5	Сначала использует, потом увеличивает

<i>Декремент (префиксный)</i>	--	int x = 5; int y = --x;	x=4, y=4	Сначала уменьшает, потом использует
<i>Декремент (постфиксный)</i>	--	int x = 5; int y = x--;	x=4, y=5	Сначала использует, потом уменьшает

Составные операторы присваивания

Операция	Оператор	Пример	Эквивалент
<i>Присвоение со сложением</i>	+=	a += 5;	a = a + 5;
<i>Присвоение с вычитанием</i>	-=	b -= 3;	b = b - 3;
<i>Присвоение с умножением</i>	*=	c *= 2;	c = c * 2;
<i>Присвоение с делением</i>	/=	d /= 4;	d = d / 4;
<i>Присвоение с остатком</i>	%=	e %= 3;	e = e % 3;

Примеры использования

```
#include <iostream>
using namespace std;

int main() {
    // Базовые арифметические операции
    int a = 15, b = 4;
    cout << "a = " << a << ", b = " << b << endl;
    cout << "a + b = " << a + b << endl; // 19
    cout << "a - b = " << a - b << endl; // 11
    cout << "a * b = " << a * b << endl; // 60
    cout << "a / b = " << a / b << endl; // 3 (целочисленное деление!)
    cout << "a % b = " << a % b << endl; // 3

    // Деление с вещественными числами
    double x = 15.0, y = 4.0;
    cout << "x / y = " << x / y << endl; // 3.75

    // Инкремент и декремент
    int counter = 5;
    cout << "counter = " << counter << endl;
    cout << "++counter = " << ++counter << endl; // 6
    cout << "counter++ = " << counter++ << endl; // 6 (потом станет 7)
    cout << "counter = " << counter << endl; // 7

    // Составные операторы
    int num = 10;
    num += 3; // num = 13
    num -= 2; // num = 11
    num *= 2; // num = 22
    num /= 4; // num = 5
    num %= 3; // num = 2

    return 0;
}
```

Приоритет операций

1. () - скобки
2. ++, -- - инкремент/декремент
3. *, /, % - умножение, деление, остаток
4. +, - - сложение, вычитание
5. =, +=, -= и т.д. - присваивание

6. Стандартные функции и операции

Математические функции (<cmath>)

Функция	Описание	Пример	Результат
sqrt(x)	Квадратный корень	sqrt(16)	4.0
pow(x, y)	Возведение в степень	pow(2, 3)	8.0
abs(x)	Модуль целого числа	abs(-5)	5
fabs(x)	Модуль вещественного числа	fabs(-3.14)	3.14
ceil(x)	Округление вверх	ceil(3.2)	4.0
floor(x)	Округление вниз	floor(3.8)	3.0
round(x)	Математическое округление	round(3.5)	4.0
sin(x)	Синус (в радианах)	sin(M_PI/2)	1.0
cos(x)	Косинус (в радианах)	cos(0)	1.0
tan(x)	Тангенс (в радианах)	tan(M_PI/4)	1.0
exp(x)	Экспонента (e^x)	exp(1)	2.718
log(x)	Натуральный логарифм	log(10)	2.302
log10(x)	Десятичный логарифм	log10(100)	2.0
fmod(x, y)	Остаток от деления вещественных	fmod(5.7, 2.0)	1.7

Функции для символов (<cctype>)

Функция	Описание	Пример	Результат
isalpha(c)	Является ли буквой	isalpha('A')	true (≠0)
isdigit(c)	Является ли цифрой	isdigit('5')	true (≠0)
isalnum(c)	Буква или цифра	isalnum('A')	true (≠0)
islower(c)	Строчная буква	islower('a')	true (≠0)
isupper(c)	Заглавная буква	isupper('A')	true (≠0)
isspace(c)	Пробельный символ	isspace(' ')	true (≠0)
toupper(c)	В верхний регистр	toupper('a')	'A'
tolower(c)	В нижний регистр	tolower('B')	'b'

Строковые функции (<string>)

Функция	Описание	Пример	Результат
length()	Длина строки	str.length()	Длина строки
size()	Размер строки	str.size()	Размер строки
empty()	Пустая строка?	str.empty()	true/false
append(str)	Добавление строки	s1.append(s2)	Объединение
find(str)	Поиск подстроки	s.find("abc")	Позиция или string::npos
substr(pos, len)	Извлечение подстроки	s.substr(0, 5)	Подстрока
replace(pos, len, str)	Замена подстроки	s.replace(0, 1, "X")	Измененная строка
erase(pos, len)	Удаление подстроки	s.erase(0, 5)	Укороченная строка
c_str()	C-строка (const char*)	s.c_str()	Указатель на массив char

Операции сравнения

Операция	Описание	Пример	Результат
==	Равно	a == b	true/false
!=	Не равно	a != b	true/false
<	Меньше	a < b	true/false
>	Больше	a > b	true/false
<=	Меньше или равно	a <= b	true/false
>=	Больше или равно	a >= b	true/false

Логические операции

Операция	Описание	Пример	Результат
&&	Логическое И	a && b	true если оба true
	Логическое ИЛИ	a b	true если хотя бы один true
!	Логическое НЕ	!a	Противоположное значение

Битовые операции

Операция	Описание	Пример	Результат
&	Побитовое И	5 & 3	1 (101 & 011 = 001)
	Побитовое ИЛИ	5 3	7 (101 011 = 111)
^	Побитовое XOR	5 ^ 3	6 (101 ^ 011 = 110)
~	Побитовое НЕ	~5	Инвертированные биты
<<	Сдвиг влево	5 << 1	10 (101 → 1010)
>>	Сдвиг вправо	5 >> 1	2 (101 → 10)

Константы из <cmath>

Константа	Описание	Значение
M_PI	Число π	3.141592653589793
M_E	Число e	2.718281828459045
M_LOG2E	$\log_2(e)$	1.4426950408889634
M_LOG10E	$\log_{10}(e)$	0.4342944819032518

Задачи для самостоятельной работы

- Дан радиус окружности, подсчитать длину окружности.
- Дан радиус окружности, подсчитать площадь круга.
- Дан произвольный треугольник. Известны стороны a и b и угол между ними x . Найти третью сторону c .
- Дан прямоугольный треугольник с катетами a и b . Найти гипотенузу c .
- Дан произвольный треугольник со сторонами a , b и c . Найти площадь треугольника.
- Найти среднеарифметическое 2-х чисел.
- Вычислить объём шара радиуса R .
- Найти расстояние между двумя точками с данными координатами.
- Найти среднее арифметическое трёх заданных чисел.
- По ребру найти площадь грани, площадь боковой поверхности и объём куба.
- Для заданного целого числа a напечатать следующую таблицу:

a
a^3
a^6
$a^6 a^3 a$
- Даны два действительных числа a и b . Получить их сумму, разность и произведение.
- Дана длина ребра куба. Найти объём куба и площадь его боковой поверхности.
- Даны два действительных положительных числа. Найти среднеарифметическое и среднегеометрическое этих чисел.
- Даны катеты прямоугольного треугольника. Найти его гипотенузу и площадь.
- Дана сторона равностороннего треугольника. Найти площадь этого треугольника.
- Найти среднегеометрическое 2-х чисел.
- Даны гипотенуза и катет прямоугольного треугольника. Найти катет и радиус вписанной окружности.
- Найти сумму членов арифметической прогрессии по данным значениям: a , d , n .
- Треугольник задан длинами сторон. Найти длины высот.
- Треугольник задан длинами сторон. Найти длины биссектрис.
- Треугольник задан длинами сторон. Найти длины медиан.
- Треугольник задан длинами сторон. Найти радиусы вписанной и описанной окружностей.
- Вычислить расстояние между двумя точками с координатами x_1, y_1 и x_2, y_2 .
- Дано действительное число x . Получить дробную часть x ; затем число x , округлённое до ближайшего целого; затем число без дробных цифр.
- Треугольник задан координатами своих вершин. Найти периметр и площадь треугольника.

Практическая работа №2.

Условный оператор. Оператор многочисленного ветвления в СИ++

Условные операторы - основа **управления потоком выполнения** программы. Они позволяют программе "принимать решения", делая код интеллектуальным и адаптивным.

1. Базовый условный оператор if

Простая форма:

```
if (условие) {  
    // код выполняется, если условие истинно (true)  
}
```

Пример:

```
int x = 10;  
if (x > 5) {  
    cout << "x больше 5" << endl;  
}
```

2. Оператор if-else

Полная форма:

```
if (условие) {  
    // код выполняется, если условие истинно  
} else {  
    // код выполняется, если условие ложно (false)  
}
```

Пример:

```
int age = 17;  
if (age >= 18) {  
    cout << "Совершеннолетний" << endl;  
} else {  
    cout << "Несовершеннолетний" << endl;  
}
```

3. Цепочка условий else if

Множественное ветвление:

```
if (условие1) {  
    // код для условия1  
} else if (условие2) {  
    // код для условия2  
} else if (условие3) {  
    // код для условия3  
} else {  
    // код, если ни одно условие не выполнилось  
}
```

Пример с оценками:

```
int score = 85;  
char grade;  
  
if (score >= 90) {  
    grade = 'A';  
} else if (score >= 80) {  
    grade = 'B';  
} else if (score >= 70) {  
    grade = 'C';  
} else if (score >= 60) {  
    grade = 'D';  
} else {  
    grade = 'F';  
}  
  
cout << "Оценка: " << grade << endl;
```

4. Оператор множественного ветвления switch

Синтаксис:

```
switch (выражение) {  
    case значение1:  
        // код для значение1  
        break;  
    case значение2:  
        // код для значение2  
        break;  
    case значение3:  
        // код для значение3  
        break;  
    default:  
        // код по умолчанию  
        break;  
}
```

Пример с днями недели:

```
int day = 3;  
  
switch (day) {  
    case 1:  
        cout << "Понедельник" << endl;  
        break;  
    case 2:  
        cout << "Вторник" << endl;  
        break;  
    case 3:  
        cout << "Среда" << endl;  
        break;  
    case 4:  
        cout << "Четверг" << endl;  
        break;  
    case 5:  
        cout << "Пятница" << endl;  
        break;  
    case 6:  
        cout << "Суббота" << endl;  
        break;  
    case 7:  
        cout << "Воскресенье" << endl;  
        break;  
    default:  
        cout << "Неверный номер дня" << endl;  
        break;  
}
```

5. Важные особенности switch

Ключевое слово break:

- break прерывает выполнение switch
- Без break выполнение "проваливается" на следующий case

Пример без break:

```
int number = 2;  
switch (number) {  
    case 1:  
        cout << "Один" << endl;  
    case 2:  
        cout << "Два" << endl; // выполнится  
    case 3:  
        cout << "Три" << endl; // тоже выполнится!  
    default:  
        // ...  
}
```

```
    cout << "Другое" << endl; //и это выполнится!  
}
```

// Вывод: "Два", "Три", "Другое"

Группировка case:

```
char grade = 'B';  
switch (grade) {  
    case 'A':  
    case 'B':  
    case 'C':  
        cout << "Сдал" << endl;  
        break;  
    case 'D':  
    case 'F':  
        cout << "Не сдал" << endl;  
        break;  
    default:  
        cout << "Неверная оценка" << endl;  
}
```

6. Тернарный оператор

Краткая форма if-else:

условие ? выражение_если_истина : выражение_если_ложь

Примеры:

```
int a = 10, b = 20;  
int max = (a > b) ? a : b; // max = 20
```

```
string result = (score >= 60) ? "Сдал" : "Не сдал";
```

7. Практический пример

```
#include <iostream>  
using namespace std;
```

```
int main() {  
    int choice;  
    double num1, num2, result;  
  
    cout << "Калькулятор:" << endl;  
    cout << "1. Сложение" << endl;  
    cout << "2. Вычитание" << endl;  
    cout << "3. Умножение" << endl;  
    cout << "4. Деление" << endl;  
    cout << "Выберите операцию: ";  
    cin >> choice;  
  
    cout << "Введите два числа: ";  
    cin >> num1 >> num2;  
  
    switch (choice) {  
        case 1:  
            result = num1 + num2;  
            cout << "Результат: " << result << endl;  
            break;  
        case 2:  
            result = num1 - num2;  
            cout << "Результат: " << result << endl;  
            break;  
        case 3:  
            result = num1 * num2;  
            cout << "Результат: " << result << endl;  
            break;  
        case 4:  
            if (num2 != 0) {  
                result = num1 / num2;  
            }  
    }
```

```

        cout << "Результат: " << result << endl;
    } else {
        cout << "Ошибка: деление на ноль!" << endl;
    }
    break;
default:
    cout << "Неверный выбор операции!" << endl;
}

return 0;
}

```

Ключевые различия if-else и switch:

if-else	switch
Проверяет логические условия	Проверяет равенство значения
Работает с любыми типами данных	Работает с целыми и перечислениями
Гибкость условий	Ограниченный синтаксис
Медленнее при многих условиях	Быстрее при многих вариантах

Выбор между if-else и switch зависит от конкретной задачи и типов проверяемых условий.

Задачи для самостоятельной работы (Все задачи решать через условные операторы)

1. Найти max значение из трёх величин, определяемых арифметическими выражениями $a = \text{tg}(x)$, $b = \text{ctg}(x)$, $c = \ln(x)$ при заданных значениях x , используя оператор if.
2. Даны действительные числа a , b , c . Проверить выполняется ли неравенство $c < b < a$.
3. Даны действительные числа a , b , c . Утроить эти числа, если $a > b > c$ и заменить их абсолютными значениями, если это не так.
4. Даны два действительных числа. Заменить первое число нулём, если оно меньше или равно второму, и оставить числа без изменения иначе.
5. Даны действительные числа x , y . Меньшее из этих двух чисел заменить их полусуммой, а большее – их удвоенным произведением.
6. Даны действительные числа a , b , c , d . Если $a < b < c < d$, то каждое число заменить наибольшим из них; если $a > b > c > d$, то числа оставить без изменения; иначе все числа заменяются их квадратами.
7. Даны действительные числа a , b , c . Выяснить, имеет ли уравнение $ax^2+bx+c=0$ действительные корни. Если действительные корни имеются, то найти их. В противном случае ответом должно служить сообщение, что действительных корней нет.
8. Даны действительные положительные числа a , b , c , x , y . Выяснить, пройдёт ли кирпич с рёбрами a , b , c в прямоугольное отверстие со сторонами x и y . Просовывать кирпич в отверстие разрешается только так, чтобы каждое из рёбер было параллельно или перпендикулярно каждой из сторон отверстия.
9. Даны два действительных числа. Вывести первое число, если оно больше второго, и оба числа, если это не так.
10. Даны три действительных числа. Выбрать из них те, которые принадлежат интервалу $(1,3)$.
11. Даны три действительных числа. Выбрать из них те, которые принадлежат интервалу $(4,6)$.
12. Даны три действительные числа. Возвести в квадрат те из них, значения которых не отрицательны.
13. Если сумма трёх попарно различных действительных чисел x, y, z меньше единицы, то наименьшее из этих трёх чисел заменить полусуммой двух других; иначе заменить меньшее из x и y полусуммой двух оставшихся значений.
14. Даны два числа. Если первое число больше второго по абсолютной величине, то необходимо уменьшить первое в 5 раз, иначе оставить числа без изменения.
15. Даны действительные положительные числа x , y , z . Выяснить, существует ли треугольник с длинами сторон x , y , z .
16. Составить программу определения большей площади из двух фигур: круга или квадрата. Известно, что сторона квадрата равна a , радиус круга равен r . Вывести и напечатать значение площади большей фигуры.

17. Определить, является ли целое число чётным.
18. Определить, верно ли, что при делении неотрицательного целого числа a на положительное целое число b , получается остаток равный одному из двух заданных чисел r или s .
19. Даны две точки $A(x_1, y_1)$ и $B(x_2, y_2)$. Составить программу, определяющую, которая из точек находится ближе к началу координат.
20. На плоскости XOY задана своими координатами точка A . Указать, где она расположена (на какой оси или в каком координатном угле).
21. Написать программу нахождения суммы большего и меньшего из трех чисел.

Практическая работа №3.

Операторы повтора.

Циклы позволяют **автоматизировать повторяющиеся действия**, обрабатывать коллекции данных и реализовывать итерационные алгоритмы.

1. Виды циклов в C++

Основные типы:

1. **for** - цикл с счетчиком
2. **while** - цикл с предусловием
3. **do-while** - цикл с постусловием
4. **range-based for** - цикл по коллекциям (C++11)

2. Цикл for

Базовый синтаксис:

```
for (инициализация; условие; итерация) {  
    // тело цикла  
}
```

Теория выполнения:

1. **Инициализация** - выполняется один раз перед началом
2. **Проверка условия** - если false → выход из цикла
3. **Тело цикла** - выполняется если условие true
4. **Итерация** - выполняется после тела
5. **Возврат к пункту 2**

Примеры:

```
// Классический счетчик  
for (int i = 0; i < 5; i++) {  
    cout << "i = " << i << endl;  
}
```

```
// Обратный счет  
for (int i = 10; i > 0; i--) {  
    cout << i << " ";  
}  
cout << "Старт!" << endl;
```

```
// С шагом 2  
for (int i = 0; i <= 10; i += 2) {  
    cout << i << " ";  
}
```

3. Цикл while

Синтаксис:

```
while (условие) {  
    // тело цикла  
}
```

Теория выполнения:

- Условие проверяется **ПЕРЕД** каждой итерацией

- Если условие false с самого начала - тело не выполнится ни разу

Примеры:

// Счетчик через while

```
int i = 0;
while (i < 5) {
    cout << "i = " << i << endl;
    i++;
}
```

// Чтение до определенного условия

```
int number;
while (true) { // бесконечный цикл
    cout << "Введите число (0 для выхода): ";
    cin >> number;
    if (number == 0) break;
    cout << "Квадрат: " << number * number << endl;
}
```

// Обработка пользовательского ввода

```
string input;
while (input != "exit") {
    cout << "Введите команду: ";
    cin >> input;
    cout << "Вы ввели: " << input << endl;
}
```

4. Цикл do-while

Синтаксис:

```
do {
    // тело цикла
} while (условие);
```

Теория выполнения:

- Тело выполняется **ПЕРЕД** проверкой условия
- Гарантирует **минимум одну итерацию**

Примеры:

// Меню программы

```
int choice;
do {
    cout << "1. Начать игру" << endl;
    cout << "2. Настройки" << endl;
    cout << "3. Выход" << endl;
    cout << "Выберите: ";
    cin >> choice;

    switch (choice) {
        case 1: cout << "Игра начинается!" << endl; break;
        case 2: cout << "Открыты настройки" << endl; break;
        case 3: cout << "Выход..." << endl; break;
        default: cout << "Неверный выбор!" << endl;
    }
} while (choice != 3);
```

// Валидация ввода

```
int age;
do {
    cout << "Введите возраст (1-120): ";
    cin >> age;
} while (age < 1 || age > 120);
```

5. Управление выполнением циклов

Операторы управления:

- **break** - немедленный выход из цикла
- **continue** - переход к следующей итерации
- **return** - выход из функции (и цикла)

Примеры:

// break - поиск элемента

```
for (int i = 0; i < 100; i++) {  
    if (i * i == 256) {  
        cout << "Найдено: " << i << endl;  
        break; // выход при нахождении  
    }  
}
```

// continue - пропуск нечетных чисел

```
for (int i = 1; i <= 10; i++) {  
    if (i % 2 != 0) {  
        continue; // пропустить нечетные  
    }  
    cout << i << " "; // выводятся только четные  
}
```

// Комбинирование

```
for (int i = 0; i < 10; i++) {  
    if (i == 2) continue; // пропустить 2  
    if (i == 7) break;    // выйти на 7  
    cout << i << " ";  
}
```

// Вывод: 0 1 3 4 5 6

6. Вложенные циклы

Теория:

- Каждый внутренний цикл выполняется **полностью** для каждой итерации внешнего
- Сложность: $O(n \times m)$

Примеры:

// Таблица умножения

```
for (int i = 1; i <= 10; i++) {  
    for (int j = 1; j <= 10; j++) {  
        cout << i * j << "\t";  
    }  
    cout << endl;  
}
```

// Обработка матрицы

```
const int ROWS = 3, COLS = 3;  
int matrix[ROWS][COLS] = {{1,2,3}, {4,5,6}, {7,8,9}};
```

```
for (int i = 0; i < ROWS; i++) {  
    for (int j = 0; j < COLS; j++) {  
        cout << matrix[i][j] << " ";  
    }  
    cout << endl;  
}
```

7. Выбор подходящего цикла

Критерии выбора:

Цикл	Когда использовать
for	Известно количество итераций
while	Условие выхода неизвестно заранее
do-while	Минимум одна итерация гарантирована
range-based for	Итерация по коллекциям

Задачи для самостоятельной работы

Часть 1.

1. Написать программу, которая выводит на экран ваши имя и фамилию нужное количество раз.
2. Найти сумму квадратов чисел от m до n .
3. Найти сумму квадратов нечётных чисел в интервале, заданном значениями переменных m и n .
4. Найти сумму квадратов чётных чисел в интервале, заданном значениями переменных m и n .
5. Найти сумму целых положительных чисел, кратных 4 и меньших 100.
6. Вывести на экран числовой ряд действительных чисел от m до n с шагом 0,2.
7. Написать программу, которая выводит таблицу квадратов первых n целых положительных чисел.
8. Дано целое число a и натуральное (целое неотрицательное) число n . Вычислить a^n . Решить можно, умножая a на себя n раз в цикле.
9. Составить программу, печатающую квадраты всех натуральных чисел от 0 до заданного натурального n .
10. Даны целые числа K и N ($N > 0$). Вывести N раз число K .
11. Даны два целых числа A и B ($A < B$). Вывести в порядке возрастания все целые числа, расположенные между A и B (включая сами числа A и B), а также количество N этих чисел.
12. Даны два целых числа A и B ($A < B$). Вывести в порядке убывания все целые числа, расположенные между A и B (не включая числа A и B), а также количество N этих чисел.
13. Даны два целых числа A и B ($A < B$). Найти сумму всех целых чисел от A до B включительно.
14. Даны два целых числа A и B ($A < B$). Найти произведение всех целых чисел от A до B включительно.
15. Даны два целых числа A и B ($A < B$). Найти среднее арифметическое всех целых чисел от A до B включительно.
16. Дано число n . В интервале от 1 до n сложить все чётные и перемножить все нечётные числа.
17. Дано число n . В интервале от 1 до n сложить все числа, которые делятся на 3 без остатка.
18. Дано число n . В интервале от 1 до n сложить все числа, которые делятся на 5 без остатка.
19. Дано число n . В интервале от 1 до n сложить все числа, которые делятся на 3 и не делятся на 2.
20. Дано число n . В интервале от 1 до n сложить все числа, которые делятся на 5 и не делятся на 3.
21. Вывести на экран числовой ряд действительных чисел от 10 до 20 с шагом 0,2.
22. Дано число n . В интервале от 1 до n сложить все числа, которые делятся на 3, и перемножить все, делящиеся на 2.

Часть 2.

Дано натуральное число n ($n < 9999$). Найти (1-6):

1. количество цифр в числе n (см. пример с нахождением суммы цифр числа).
2. последнюю и первую цифру числа (см. пример с нахождением суммы цифр числа).
3. дано число n . Найти сумму двух последних цифр числа n (см. пример с нахождением суммы цифр числа).
4. выяснить, входит ли цифра 3 в запись числа n (см. пример с нахождением суммы цифр числа).
5. Выяснить, сколько чётных цифр в числе (см. пример с нахождением суммы цифр числа).
6. Выяснить, сколько нечётных цифр в числе (см. пример с нахождением суммы цифр числа).

Вычислить (7-12)

7. $\sum_{i=1}^{100} \frac{1}{i^2} ;$

8. $\sum_{i=1}^{50} \frac{1}{i^3} ;$

9. $\sum_{i=1}^{10} \frac{1}{i!} ;$

10. $\sum_{i=1}^{128} \frac{1}{(2i)^2} ;$

11. $\prod_{i=1}^{52} \frac{i^2}{i^2 + 2i + 3} ;$

12. $\prod_{i=2}^{100} \frac{i+1}{i+2} ;$

Практическая работа №4.

Функции.

Функции - это базовые строительные блоки программ, которые инкапсулируют логику, обеспечивают повторное использование кода и улучшают читаемость.

1. Основные понятия

Различие функций и процедур:

- **Функция** - возвращает значение
- **Процедура** - не возвращает значение (void)
- В C++ формально все являются функциями

2. Синтаксис объявления и определения

Прототип (объявление):

```
тип_возврата имя_функции(параметры);
```

Определение:

```
тип_возврата имя_функции(параметры) {  
    // тело функции  
    return значение; // если не void  
}
```

3. Типы функций

Функция без параметров и возвращаемого значения:

```
// Процедура  
void printHello() {  
    cout << "Hello, World!" << endl;  
}
```

```
// Использование  
printHello();
```

Функция с параметрами без возвращаемого значения:

```
void printMessage(string message, int count) {  
    for (int i = 0; i < count; i++) {  
        cout << message << endl;  
    }  
}
```

```
// Использование  
printMessage("Hello", 3);
```

Функция с возвращаемым значением:

```
// Простая функция  
int square(int x) {  
    return x * x;  
}
```

```
// Функция с несколькими return  
int max(int a, int b) {  
    if (a > b) {
```

```

    return a;
} else {
    return b;
}
}

```

// Использование

```

int result = square(5); // 25
int bigger = max(10, 20); // 20

```

Функция с возвратом указателя:

```

int* createArray(int size) {
    int* arr = new int[size];
    for (int i = 0; i < size; i++) {
        arr[i] = i * i;
    }
    return arr;
}

```

// Использование

```

int* myArray = createArray(10);

```

4. Способы передачи параметров

Передача по значению (копирование):

```

void modifyValue(int x) {
    x = 100; // изменяется копия
}

int main() {
    int a = 5;
    modifyValue(a);
    cout << a; // выведет 5 (не изменилось)
}

```

Передача по ссылке:

```

void modifyReference(int &x) {
    x = 100; // изменяется оригинал
}

int main() {
    int a = 5;
    modifyReference(a);
    cout << a; // выведет 100 (изменилось)
}

```

Передача по указателю:

```

void modifyPointer(int *x) {
    *x = 100; // изменяется оригинал
}

int main() {
    int a = 5;
    modifyPointer(&a);
    cout << a; // выведет 100
}

```

Передача массивов:

// Массив как указатель

```

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
}

```

// С указателем

```
void printArray(int* arr, int size) {  
    for (int i = 0; i < size; i++) {  
        cout << *(arr + i) << " ";  
    }  
}
```

5. Возвращаемые значения

Различные типы возврата:

// Возврат базовых типов

```
int getInteger() { return 42; }  
double getDouble() { return 3.14; }  
bool isEven(int n) { return n % 2 == 0; }
```

// Возврат ссылки

```
int& getElement(int arr[], int index) {  
    return arr[index];  
}
```

// Возврат указателя

```
string* createString() {  
    return new string("Hello");  
}
```

// Возврат структуры

```
struct Point { int x, y; };  
Point createPoint(int x, int y) {  
    return {x, y};  
}
```

6. Перегрузка функций

Функции с одинаковым именем, но разными параметрами:

// Перегрузка по типу параметров

```
int add(int a, int b) {  
    return a + b;  
}
```

```
double add(double a, double b) {  
    return a + b;  
}
```

```
string add(string a, string b) {  
    return a + b;  
}
```

// Перегрузка по количеству параметров

```
int multiply(int a, int b) {  
    return a * b;  
}
```

```
int multiply(int a, int b, int c) {  
    return a * b * c;  
}
```

// Использование

```
cout << add(5, 10) << endl;    // 15  
cout << add(2.5, 3.7) << endl; // 6.2  
cout << add("Hello ", "World") << endl; // "Hello World"  
cout << multiply(2, 3) << endl; // 6  
cout << multiply(2, 3, 4) << endl; // 24
```

Пример:

```
#include <iostream>  
#include <string>  
#include <cmath>
```

```
using namespace std;
```

```
// Прототипы функций
```

```
void displayMenu();  
int getChoice();  
double calculateCircleArea(double radius);  
double calculateRectangleArea(double length, double width);  
double calculateTriangleArea(double base, double height);  
void printResult(double area);
```

```
int main() {  
    int choice;  
    double area;  
  
    do {  
        displayMenu();  
        choice = getChoice();  
  
        switch (choice) {  
            case 1: {  
                double radius;  
                cout << "Введите радиус: ";  
                cin >> radius;  
                area = calculateCircleArea(radius);  
                break;  
            }  
            case 2: {  
                double length, width;  
                cout << "Введите длину и ширину: ";  
                cin >> length >> width;  
                area = calculateRectangleArea(length, width);  
                break;  
            }  
            case 3: {  
                double base, height;  
                cout << "Введите основание и высоту: ";  
                cin >> base >> height;  
                area = calculateTriangleArea(base, height);  
                break;  
            }  
            case 4:  
                cout << "Выход..." << endl;  
                break;  
            default:  
                cout << "Неверный выбор!" << endl;  
        }  
  
        if (choice >= 1 && choice <= 3) {  
            printResult(area);  
        }  
  
    } while (choice != 4);  
  
    return 0;  
}
```

```
// Определения функций
```

```
void displayMenu() {  
    cout << "\n=== КАЛЬКУЛЯТОР ПЛОЩАДЕЙ ===" << endl;  
    cout << "1. Площадь круга" << endl;  
    cout << "2. Площадь прямоугольника" << endl;  
    cout << "3. Площадь треугольника" << endl;  
    cout << "4. Выход" << endl;  
    cout << "Выберите опцию: ";  
}
```

```
int getChoice() {  
    int choice;
```

```

    cin >> choice;
    return choice;
}

double calculateCircleArea(double radius) {
    return M_PI * radius * radius;
}

double calculateRectangleArea(double length, double width) {
    return length * width;
}

double calculateTriangleArea(double base, double height) {
    return 0.5 * base * height;
}

void printResult(double area) {
    cout << "Площадь: " << area << endl;
}

```

Задачи для самостоятельной работы

Список задач (в каждой нужно создать процедуру или функцию)

1. Даны действительные числа s, t . Получить: $f(t, -2s, 1.17) + f(2, 2, t, s - t)$, где $f(a, b, c) = \frac{2a - b - \sin(c)}{5 + |c|}$
2. Даны действительные числа s, t . Получить: $f(2t, 3s, 5) + f(t, -t, s + t)$, где $f(a, b, c) = \frac{2a + b + \sin(c)}{7.5 - c}$
3. Даны действительные числа s, t . Получить: $f(t, s + t, -7) + f(3, s - 10.5s)$, где $f(a, b, c) = \frac{a + 2b + \ln(c)}{a + c}$
4. Даны действительные числа s, t . Получить: $g(1/2, s) + g(t, s) - g(2s - 1, st)$, где $g(a, b) = \frac{a^2 + b^2}{a^2 + 2ab + 3b^2 + 4}$
5. Даны действительные числа s, t . Получить: $g(1/3, s + t) + g(t, s) - g(2s, st)$, где $g(a, b) = \frac{a^3 + b^3}{a^2 + 3ab + 4b^3 - 5}$
6. Даны действительные числа a, b, c . Получить: $\frac{\max(a, a + b) + \max(a, b + c)}{1 + \max(a + bc, 1, 15)}$
7. Даны действительные числа a, b, c . Получить: $\frac{\max(a, a * b) + \max(c, \frac{b}{c})}{8 + \max(c + ba, 3, 25)}$
8. Даны действительные числа a, b, c . Получить: $\frac{\min(c, a - b) - \min(b, a + 2c)}{100 - \min(a + c, 2b)}$
9. Даны действительные числа a, b, c . Получить: $u = \min(a, b), v = \min(a, b), z = \min(u + v^2, 3, 14)$
10. Даны действительные числа a, b, c . Получить: $u = \min(a, b, c), v = \min(a, b, c), z = \min(u^2 + v^2, u * v)$
11. Даны действительные числа a, b, c . Получить: $u = \min(a, b, c), v = \min(2a + 2b + 2c, ab + c), z = \min(u^2 + v, u * v)$
12. Напишите функцию возведения в степень по формуле: $a^n = \text{Exp}(\text{Ln}(a) * n)$. Используйте ее в программе для возведения в нужную степень введенного с клавиатуры числа.
13. Дано натуральное число n . Среди чисел $1, 2, 3, \dots, n$ найти все те, которые можно представить в виде сумм квадратов двух натуральных чисел. Определить процедуру, позволяющую распознавать полные квадраты.
14. Напишите программу поиска максимального из четырех чисел с использованием подпрограммы поиска большего из двух.
15. Дано число n . Вычислите сумму: $1! + 2! + 3! + \dots + n!$, создав функцию вычисления факториала числа.
16. Создайте программу для подсчета числа четных цифр, используемых в записи введенного числа M .
17. Создайте программу для подсчета числа нечетных цифр, используемых в записи введенного числа M .

18. Создайте программу вычисления суммы трехзначных чисел, в десятичной записи которых нет четных цифр.
19. Создайте программу вычисления суммы трехзначных чисел, в десятичной записи которых нет нечетных цифр.
20. Вычислить площадь правильного шестиугольника со стороной a , используя подпрограмму вычисления площади треугольника.
21. Составить программу, определяющую, в каком из данных двух чисел больше цифр (создать подпрограмму для вычисления количества цифр в числе).
22. Дано натуральное число n . Выяснить, можно ли представить его в виде произведения трех последовательных натуральных чисел (сделать функцию для данных целей).

Практическая работа №5.

Массивы.

Что такое массив?

Массив - это структура данных, которая хранит набор элементов **одного типа** в **последовательных ячейках памяти**.

Основные характеристики:

- Все элементы имеют одинаковый тип
- Размер массива фиксирован при объявлении
- Индексация начинается с 0
- Элементы хранятся в смежных ячейках памяти

2. Одномерные массивы

Объявление и инициализация

// Способ 1: Объявление без инициализации

```
int arr1[5]; // Массив из 5 целых чисел (значения мусор)
```

// Способ 2: Объявление с инициализацией

```
int arr2[5] = {1, 2, 3, 4, 5};
```

// Способ 3: Автоматическое определение размера

```
int arr3[] = {10, 20, 30, 40}; // Размер = 4
```

// Способ 4: Частичная инициализация

```
int arr4[5] = {1, 2}; // Первые два элемента: 1, 2, остальные: 0
```

// Способ 5: Инициализация нулями

```
int arr5[5] = {0}; // Все элементы = 0
```

Доступ к элементам

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int numbers[5] = {10, 20, 30, 40, 50};
```

// Обращение к элементам по индексу

```
    cout << "numbers[0] = " << numbers[0] << endl; // 10
```

```
    cout << "numbers[2] = " << numbers[2] << endl; // 30
```

```
    cout << "numbers[4] = " << numbers[4] << endl; // 50
```

// Изменение элемента

```
    numbers[1] = 25;
```

```
    cout << "numbers[1] = " << numbers[1] << endl; // 25
```

// НЕПРАВИЛЬНО: выход за границы массива!

// numbers[5] = 60; // ОШИБКА! Неопределенное поведение

```
    return 0;
```

```
}
```

Перебор элементов массива

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int arr[] = {5, 10, 15, 20, 25};
```

```
    int size = sizeof(arr) / sizeof(arr[0]); // Определение размера
```

```
    cout << "Array elements:" << endl;
```

```

// Способ 1: for с индексом
for (int i = 0; i < size; i++) {
    cout << "arr[" << i << "] = " << arr[i] << endl;
}

cout << "\nIn reverse order:" << endl;

// Способ 2: обратный порядок
for (int i = size - 1; i >= 0; i--) {
    cout << arr[i] << " ";
}
cout << endl;

// Способ 3: range-based for (C++11 и выше)
cout << "\nUsing range-based for:" << endl;
for (int element : arr) {
    cout << element << " ";
}
cout << endl;

return 0;
}

```

Практические примеры

```

#include <iostream>
using namespace std;

int main() {
    // Пример 1: Сумма элементов массива
    int nums[] = {1, 2, 3, 4, 5};
    int sum = 0;

    for (int i = 0; i < 5; i++) {
        sum += nums[i];
    }
    cout << "Sum = " << sum << endl;

    // Пример 2: Поиск максимального элемента
    int values[] = {34, 12, 78, 23, 45};
    int max = values[0];

    for (int i = 1; i < 5; i++) {
        if (values[i] > max) {
            max = values[i];
        }
    }
    cout << "Max value = " << max << endl;

    // Пример 3: Заполнение массива с клавиатуры
    const int SIZE = 3;
    double grades[SIZE];

    cout << "\nEnter " << SIZE << " grades:" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << "Grade " << i + 1 << ": ";
        cin >> grades[i];
    }

    cout << "\nEntered grades:" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << grades[i] << " ";
    }
    cout << endl;

    return 0;
}

```

3. Многомерные массивы

Двумерные массивы (матрицы)

```
#include <iostream>
using namespace std;

int main() {
    // Объявление и инициализация
    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    // Альтернативная инициализация
    int matrix2[2][4] = {1, 2, 3, 4, 5, 6, 7, 8};

    cout << "Matrix elements:" << endl;

    // Перебор элементов матрицы
    for (int i = 0; i < 3; i++) { // строки
        for (int j = 0; j < 3; j++) { // столбцы
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    // Пример: сумма строк
    cout << "\nSum of each row:" << endl;
    for (int i = 0; i < 3; i++) {
        int rowSum = 0;
        for (int j = 0; j < 3; j++) {
            rowSum += matrix[i][j];
        }
        cout << "Row " << i << " sum = " << rowSum << endl;
    }

    return 0;
}
```

Пример: таблица умножения

```
#include <iostream>
#include <iomanip> // для setw()
using namespace std;

int main() {
    const int ROWS = 10;
    const int COLS = 10;
    int multiplication[ROWS][COLS];

    // Заполнение таблицы умножения
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            multiplication[i][j] = (i + 1) * (j + 1);
        }
    }

    // Вывод таблицы
    cout << "Multiplication table:" << endl;
    cout << setw(4) << " ";
    for (int j = 0; j < COLS; j++) {
        cout << setw(4) << j + 1;
    }
    cout << endl;
}
```

```

for (int i = 0; i < ROWS; i++) {
    cout << setw(4) << i + 1;
    for (int j = 0; j < COLS; j++) {
        cout << setw(4) << multiplication[i][j];
    }
    cout << endl;
}

return 0;
}

```

4. Символьные массивы (строки в стиле C)

```

#include <iostream>
#include <cstring> // для функций работы со строками C
using namespace std;

```

```

int main() {
    // Строка как массив символов
    char str1[] = "Hello"; // Автоматически добавляет '\0'
    char str2[6] = {'W', 'o', 'r', 'l', 'd', '\0'};
    char str3[20];

    // Ввод строки
    cout << "Enter a string (max 19 chars): ";
    cin.getline(str3, 20);

    // Вывод строк
    cout << "str1: " << str1 << endl;
    cout << "str2: " << str2 << endl;
    cout << "str3: " << str3 << endl;

    // Функции из <cstring>
    cout << "\nString functions:" << endl;
    cout << "Length of str1: " << strlen(str1) << endl;

    char copy[20];
    strcpy(copy, str1); // Копирование
    cout << "Copy: " << copy << endl;

    strcat(copy, " "); // Конкатенация
    strcat(copy, str2);
    cout << "Concatenated: " << copy << endl;

    // Сравнение строк
    if (strcmp(str1, "Hello") == 0) {
        cout << "str1 equals \"Hello\"" << endl;
    }

    return 0;
}

```

5. Передача массивов в функции

```

#include <iostream>
using namespace std;

// Функция для вывода массива
void printArray(int arr[], int size) {
    cout << "Array elements: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

```

```
// Функция для изменения массива
void doubleArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] *= 2; // Изменения сохраняются!
    }
}
```

```
// Функция с двумерным массивом
void printMatrix(int matrix[][3], int rows) {
    cout << "Matrix:" << endl;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 3; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
}
```

```
int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};

    printArray(numbers, 5);
    doubleArray(numbers, 5);
    cout << "After doubling:" << endl;
    printArray(numbers, 5);

    printMatrix(matrix, 2);

    return 0;
}
```

6. Общие задачи с массивами

```
#include <iostream>
#include <algorithm> // для sort()
using namespace std;

int main() {
    const int SIZE = 7;
    int arr[SIZE] = {23, 5, 67, 12, 89, 34, 42};

    // Сортировка массива (по возрастанию)
    sort(arr, arr + SIZE);

    cout << "Sorted array: ";
    for (int i = 0; i < SIZE; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Поиск элемента
    int searchValue;
    cout << "\nEnter value to search: ";
    cin >> searchValue;

    bool found = false;
    for (int i = 0; i < SIZE; i++) {
        if (arr[i] == searchValue) {
            cout << "Value found at index " << i << endl;
            found = true;
            break;
        }
    }

    if (!found) {
        cout << "Value not found" << endl;
    }
}
```

```

// Подсчет четных чисел
int evenCount = 0;
for (int i = 0; i < SIZE; i++) {
    if (arr[i] % 2 == 0) {
        evenCount++;
    }
}
cout << "Even numbers: " << evenCount << endl;

// Реверс массива
cout << "Reversed array: ";
for (int i = SIZE - 1; i >= 0; i--) {
    cout << arr[i] << " ";
}
cout << endl;

return 0;
}

```

Динамические массивы в C++

1. Основы динамической памяти

Что такое динамические массивы?

Динамические массивы - это массивы, память для которых выделяется во время выполнения программы (время выполнения), а не во время компиляции.

Преимущества динамических массивов:

- Размер можно определить во время выполнения
- Память выделяется и освобождается по необходимости
- Можно изменять размер массива (перераспределение памяти)

2. Операторы **new** и **delete**

Выделение памяти для одного элемента

```

#include <iostream>
using namespace std;

int main() {
    // Выделение памяти для одного int
    int* ptr = new int;
    *ptr = 42;
    cout << "Value: " << *ptr << endl;

    // Освобождение памяти
    delete ptr;
    ptr = nullptr; // Хорошая практика

    return 0;
}

```

Выделение памяти для массива

```

#include <iostream>
using namespace std;

int main() {
    int size;

    // Запрашиваем размер у пользователя
    cout << "Enter array size: ";
    cin >> size;

    // Выделение памяти для массива
    int* dynamicArray = new int[size];
}

```

```

// Заполнение массива
cout << "Enter " << size << " numbers:" << endl;
for (int i = 0; i < size; i++) {
    cout << "Element " << i << ": ";
    cin >> dynamicArray[i];
}

// Вывод массива
cout << "\nArray elements:" << endl;
for (int i = 0; i < size; i++) {
    cout << "dynamicArray[" << i << "] = " << dynamicArray[i] << endl;
}

// ОЧЕНЬ ВАЖНО: освобождение памяти
delete[] dynamicArray;
dynamicArray = nullptr;

return 0;
}

```

3. Создание и работа с динамическими массивами

Пример 1: Базовые операции

```

#include <iostream>
using namespace std;

int main() {
    // Создание динамического массива
    int n;
    cout << "Enter array size: ";
    cin >> n;

    double* prices = new double[n];

    // Инициализация массива
    for (int i = 0; i < n; i++) {
        prices[i] = (i + 1) * 10.5;
    }

    // Работа с массивом
    double sum = 0;
    for (int i = 0; i < n; i++) {
        cout << "prices[" << i << "] = " << prices[i] << endl;
        sum += prices[i];
    }

    cout << "\nAverage price: " << sum / n << endl;

    // Освобождение памяти
    delete[] prices;
    prices = nullptr;

    return 0;
}

```

Пример 2: Матрица (двумерный динамический массив)

```

#include <iostream>
using namespace std;

int main() {
    int rows, cols;

    cout << "Enter number of rows: ";
    cin >> rows;
    cout << "Enter number of columns: ";
    cin >> cols;

    // Создание двумерного динамического массива
    int** matrix = new int*[rows]; // Массив указателей на строки
}

```

```

for (int i = 0; i < rows; i++) {
    matrix[i] = new int[cols]; // Каждая строка - это массив
}

// Заполнение матрицы
int counter = 1;
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        matrix[i][j] = counter++;
    }
}

// Вывод матрицы
cout << "\nMatrix:" << endl;
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        cout << matrix[i][j] << "t";
    }
    cout << endl;
}

// Освобождение памяти (ВАЖНО: в обратном порядке!)
for (int i = 0; i < rows; i++) {
    delete[] matrix[i]; // Освобождаем каждую строку
}
delete[] matrix; // Освобождаем массив указателей
matrix = nullptr;

return 0;
}

```

4. Изменение размера динамического массива

```

#include <iostream>
using namespace std;

int main() {
    int size;
    cout << "Enter initial array size: ";
    cin >> size;

    // Создание начального массива
    int* arr = new int[size];

    cout << "Enter " << size << " elements:" << endl;
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }

    // Запрос на увеличение размера
    int newSize;
    cout << "\nEnter new size (larger than " << size << "): ";
    cin >> newSize;

    // Создание нового массива большего размера
    int* newArr = new int[newSize];

    // Копирование старых данных
    for (int i = 0; i < size; i++) {
        newArr[i] = arr[i];
    }

    // Заполнение новых элементов
    cout << "Enter " << (newSize - size) << " additional elements:" << endl;
    for (int i = size; i < newSize; i++) {
        cin >> newArr[i];
    }
}

```



```

// Освобождаем старый массив
delete[] arr;

// Перенаправляем указатель на новый массив
arr = newArr;
size = newSize;

// Вывод результата
cout << "\nResized array:" << endl;
for (int i = 0; i < size; i++) {
    cout << arr[i] << " ";
}
cout << endl;

// Освобождаем память
delete[] arr;
arr = nullptr;

return 0;
}

```

Современный C++: vector вместо динамических массивов

В современном C++ рекомендуется использовать std::vector вместо ручного управления динамическими массивами:

```

#include <iostream>
#include <vector> // Включаем заголовок vector
using namespace std;

int main() {
    // Создание вектора (динамического массива)
    vector<int> numbers;

    // Добавление элементов
    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);

    // Доступ к элементам
    cout << "First element: " << numbers[0] << endl;
    cout << "Size: " << numbers.size() << endl;

    // Автоматическое управление памятью
    numbers.push_back(40);
    numbers.push_back(50);

    // Перебор элементов
    cout << "All elements: ";
    for (int num : numbers) {
        cout << num << " ";
    }
    cout << endl;

    // Вектор сам освобождает память при выходе из области видимости

    return 0;
}

```

Преимущества vector:

- Автоматическое управление памятью
- Автоматическое изменение размера
- Безопасность (проверка границ с помощью at())
- Встроенные методы (вставка, удаление, поиск)
- Совместимость с STL алгоритмами

Массивы, заполненные случайными значениями в C++

1. Основы генерации случайных чисел

Библиотеки для работы со случайными числами

```
#include <iostream>
#include <cstdlib> // для rand(), srand(), RAND_MAX
#include <ctime>    // для time()
#include <random>    // для современных генераторов (C++11 и выше)
#include <algorithm> // для shuffle()
```

2. Классический способ (устаревший, но простой)

Пример 1: Базовое заполнение

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    // Инициализация генератора случайных чисел
    srand(time(0)); // Используем текущее время как seed

    const int SIZE = 10;
    int arr[SIZE];

    // Заполнение массива случайными числами от 0 до 99
    for (int i = 0; i < SIZE; i++) {
        arr[i] = rand() % 100; // % 100 дает диапазон 0-99
    }

    // Вывод массива
    cout << "Random array (0-99):" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Пример 2: С разными диапазонами

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    srand(time(0));
    const int SIZE = 8;
    int numbers[SIZE];

    cout << "Different ranges:" << endl;

    // 1. От 0 до RAND_MAX
    cout << "1. Range 0 to " << RAND_MAX << ":" << endl;
    for (int i = 0; i < SIZE; i++) {
        numbers[i] = rand();
        cout << numbers[i] << " ";
    }
    cout << endl << endl;

    // 2. От 0 до N-1
    cout << "2. Range 0 to 49:" << endl;
```

```

for (int i = 0; i < SIZE; i++) {
    numbers[i] = rand() % 50; // 0-49
    cout << numbers[i] << " ";
}
cout << endl << endl;

// 3. Om 1 до N
cout << "3. Range 1 to 100:" << endl;
for (int i = 0; i < SIZE; i++) {
    numbers[i] = (rand() % 100) + 1; // 1-100
    cout << numbers[i] << " ";
}
cout << endl << endl;

// 4. Om MIN до MAX
cout << "4. Range -50 to 50:" << endl;
int min = -50, max = 50;
for (int i = 0; i < SIZE; i++) {
    numbers[i] = min + (rand() % (max - min + 1));
    cout << numbers[i] << " ";
}
cout << endl << endl;

// 5. Дробные числа от 0 до 1
cout << "5. Range 0.0 to 1.0:" << endl;
double doubles[SIZE];
for (int i = 0; i < SIZE; i++) {
    doubles[i] = static_cast<double>(rand()) / RAND_MAX;
    cout << doubles[i] << " ";
}
cout << endl;

return 0;
}

```

Функция для генерации в диапазоне

```

#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

// Функция для генерации случайного числа в диапазоне [min, max]
int randomInRange(int min, int max) {
    return min + (rand() % (max - min + 1));
}

// Функция для генерации случайного дробного числа в диапазоне [min, max]
double randomDouble(double min, double max) {
    return min + (static_cast<double>(rand()) / RAND_MAX) * (max - min);
}

int main() {
    srand(time(0));

    cout << "Integers in range [10, 20]:" << endl;
    for (int i = 0; i < 5; i++) {
        cout << randomInRange(10, 20) << " ";
    }
    cout << endl;

    cout << "Doubles in range [5.0, 10.0]:" << endl;
    for (int i = 0; i < 5; i++) {
        cout << randomDouble(5.0, 10.0) << " ";
    }
    cout << endl;

    return 0;
}

```

Список задач

(все массивы динамические, заполняются рандомно)

Часть 1

1. Дан массив из n чисел. Найти наименьшее из чётных чисел массива.
2. Дан массив из n чисел. Найти наибольшее из нечётных чисел массива.
3. Дан массив из n чисел. Выяснить, имеются ли в последовательности два идущих подряд нулевых элемента.
4. Дан массив из n чисел. Выяснить, какое число встречается в массиве раньше – положительное или отрицательное. Если все элементы массива равны нулю, то сообщить об этом.
5. Дан массив из n чисел. Найти номер первого чётного элемента массива; если чётных элементов нет, то ответом должно быть число 0.
6. Дан массив из n чисел. Найти номер последнего нечётного элемента последовательности; если нечётных элементов нет, то ответом должно быть число 0.
7. Дан массив из n чисел. Вычислить произведение элементов в массиве до первого отрицательного.
8. Дан массив из n чисел. Вычислить сумму элементов в массиве после первого отрицательного.
9. Дан массив из n чисел. Вычислить произведение элементов массива до первого нуля.
10. Дан массив из n чисел. Вычислить сумму элементов после первого нуля.
11. Дан массив из n чисел и число A . Подсчитать количество элементов, больших A .
12. Дан массив из n чисел. Составить программу для вычисления суммы элементов массива, имеющих чётные индексы.
13. Дан массив из n чисел. Составить программу для вычисления произведения элементов массива, имеющих нечётные индексы.

Даны массив (A1, A2, ..., A17). Получить новый массив, состоящий из (14-16):

14. A11, A12, ..., A17, A1, A2, ..., A10;
15. A11, A12, ..., A17, A10, A9, ..., A1;
16. A1, A3, ..., A17, A2, A4, ..., A16.
17. Дан массив из n чисел. Составить программу для нахождения среднеарифметического элементов массива.
18. Дан массив из n чисел. Все четные элементы умножить на 3, а к нечетным прибавить 2.
19. Дан массив из n чисел и число A . Заменить все элементы, делящиеся на A без остатка, на -11.
20. Дан массив из n чисел. Найти минимальный элемент и вычесть его из всех остальных элементов массива.
21. Дан массив из n чисел. Поменять в массиве местами наибольший и наименьший элементы.
22. Дан массив из n чисел. Поменять в массиве местами наибольший и последний элементы.
23. Дан массив из n чисел. Поменять в массиве местами наименьший и первый элементы.

Часть 2

1. Дан двумерный числовой массив. Значения элементов главной диагонали возвести в квадрат.
2. Дан двумерный массив. Поменять местами значения элементов столбца и строки на месте стыка минимального значения массива (или первого из минимальных). Например, если индекс минимального элемента (3;1), т.е. он находится на пересечении 3 строки и 1 столбца, то 3 строку сделать 1 столбцом, а 1 столбец сделать 3 строкой.
3. Дан двумерный массив. Сформировать одномерный массив только из четных элементов двумерного массива.
4. Дан двумерный массив. Найти сумму элементов массива, начиная с элемента, индексы которого вводит пользователь, и заканчивая элементом, индексы которого вводит пользователь.
5. Дан двумерный массив. Сформировать одномерный массив только из элементов двумерного массива с четной суммой индексов.
6. Дан двумерный массив. Сделать из него 2 одномерных: в одном – четные элементы двумерного массива, в другом – нечетные.
7. Вычислить сумму и число положительных элементов матрицы $A[N, N]$, находящихся над главной диагональю.

8. В квадратной матрице определить максимальный и минимальные элементы. Если таких элементов несколько, то максимальный определяется по наибольшей сумме своих индексов, минимальный – по наименьшей.
9. Для заданной квадратной матрицы сформировать одномерный массив из ее диагональных элементов.
10. Заданы матрица порядка n и число k . Вычесть из элементов k -й строки диагональный элемент, расположенный в этой строке.
11. Заданы матрица порядка n и число k . Вычесть из элементов k -го столбца диагональный элемент, расположенный в этом столбце.
12. Дана прямоугольная матрица. Найти строку с наибольшей и наименьшей суммой элементов. Вывести на печать найденные строки и суммы их элементов.
13. Дана прямоугольная матрица. Найти столбец с наибольшей и наименьшей суммой элементов. Вывести на печать найденные столбцы и суммы их элементов.
14. Дан двумерный массив. Выяснить, в каких строках сумма элементов меньше введенного пользователем значения.
15. Дан двумерный массив. Выяснить, в каких столбцах произведение элементов меньше введенного пользователем значения.
16. Дан двумерный массив. Выяснить, есть ли столбец и строка с одинаковой суммой элементов. Если есть, напечатать их номера.
17. Дан двумерный массив. Выяснить, есть ли столбец и строка с одинаковым произведением элементов. Если есть, напечатать их номера.
18. Дан двумерный массив. Выяснить, есть ли строки с одинаковой суммой элементов. Если есть, вывести их номера.
19. Дан двумерный массив. Выяснить, есть ли столбцы с одинаковой суммой элементов. Если есть, вывести их номера.
20. Дан двумерный массив. Определить максимальный среди положительных элементов, минимальный среди отрицательных элементов и поменять их местами.
21. Дан двумерный массив. Заменить первый нуль в каждом столбце на количество нулей в этом столбце.
22. Дан двумерный массив. Заменить первый нуль в каждой строке на количество нулей в этой строке.

Практическая работа №6.

Строки.

1. Введение в строки

В C++ существует два основных способа работы со строками:

Строки в стиле C (C-strings) - массивы символов с нуль-терминатором

Класс std::string - объектно-ориентированный подход (рекомендуется)

2. Строки в стиле C (C-strings)

Базовые концепции

```
#include <iostream>
#include <cstring> // Для функций работы со строками C
using namespace std;

int main() {
    // Создание строк разными способами
    char str1[] = "Hello"; // Автоматически добавляется '\0'
    char str2[6] = {'W', 'o', 'r', 'l', 'd', '\0'};
    char str3[20]; // Пустой массив на 20 символов

    cout << "str1: " << str1 << endl; // Hello
    cout << "str2: " << str2 << endl; // World

    // Длина строки (без учета '\0')
    cout << "Length of str1: " << strlen(str1) << endl; // 5

    // Размер массива (включая '\0')
    cout << "Size of str1 array: " << sizeof(str1) << endl; // 6

    return 0;
}
```

Основные функции из <cstring>

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char str1[20] = "Hello";
    char str2[20] = "World";
    char result[50];

    // 1. Копирование строк (strcpy)
    strcpy(result, str1);
    cout << "After strcpy: " << result << endl; // Hello

    // 2. Конкатенация (strcat)
    strcat(result, " ");
    strcat(result, str2);
    cout << "After strcat: " << result << endl; // Hello World

    // 3. Сравнение строк (strcmp)
    int comparison = strcmp(str1, str2);
    if (comparison == 0) {
        cout << "Strings are equal" << endl;
    } else if (comparison < 0) {
        cout << "str1 < str2 (lexicographically)" << endl; // Hello < World
    } else {
        cout << "str1 > str2" << endl;
    }

    // 4. Поиск символа (strchr)
    char* found = strchr(str1, 'l');
```

```

if (found) {
    cout << "Found 'l' at position: " << (found - str1) << endl; // 2
}

// 5. Поиск подстроки (strstr)
char text[] = "Hello World, Hello C++";
char* substring = strstr(text, "World");
if (substring) {
    cout << "Found 'World' at position: " << (substring - text) << endl; // 6
}

return 0;
}

```

Пример: безопасное копирование

```

#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char source[] = "This is a very long string that might not fit";
    char destination[20];

    // ОПАСНО: может переполнить буфер
    // strcpy(destination, source);

    // БЕЗОПАСНО: ограниченное копирование
    strncpy(destination, source, sizeof(destination) - 1);
    destination[sizeof(destination) - 1] = '\0'; // Гарантируем завершение строки

    cout << "Destination: " << destination << endl;

    return 0;
}

```

3. Класс std::string (рекомендуемый способ)

Базовые операции

```

#include <iostream>
#include <string> // Для std::string
using namespace std;

int main() {
    // Создание строк
    string s1 = "Hello";
    string s2("World");
    string s3(5, 'A'); // "AAAAA"
    string s4; // Пустая строка

    // Проверка на пустоту
    cout << "Is s4 empty? " << (s4.empty() ? "Yes" : "No") << endl; // Yes

    // Длина строки
    cout << "Length of s1: " << s1.length() << endl; // 5
    cout << "Size of s1: " << s1.size() << endl; // 5 (синоним length)

    // Доступ к символам
    cout << "First character: " << s1[0] << endl; // H
    cout << "Last character: " << s1[s1.length()-1] << endl; // o

    // Безопасный доступ с проверкой границ
    try {
        cout << "Character at position 10: " << s1.at(10) << endl;
    } catch (const out_of_range& e) {
        cout << "Error: " << e.what() << endl;
    }
}

```

```
    return 0;
}
```

Конкатенация и присваивание

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s1 = "Hello";
    string s2 = "World";

    // Конкатенация
    string s3 = s1 + " " + s2 + "!";
    cout << "s3: " << s3 << endl; // Hello World!

    // Конкатенация с другими типами
    string s4 = "Number: " + to_string(42) + ", Pi: " + to_string(3.14159);
    cout << "s4: " << s4 << endl;

    // Добавление в конец
    s1.append(" ");
    s1.append(s2);
    cout << "After append: " << s1 << endl;

    // Добавление символов
    s2.push_back('!');
    cout << "s2 after push_back: " << s2 << endl; // World!

    // Присваивание
    string s5;
    s5.assign("New string");
    cout << "s5: " << s5 << endl;

    s5.assign(10, '-');
    cout << "s5 after assign: " << s5 << endl; // -----

    return 0;
}
```

Сравнение строк

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s1 = "apple";
    string s2 = "banana";
    string s3 = "Apple";
    string s4 = "apple";

    // Сравнение
    if (s1 == s4) {
        cout << "s1 equals s4" << endl; // Выполнится
    }

    if (s1 < s2) {
        cout << "s1 < s2 (lexicographically)" << endl; // Выполнится
    }

    if (s1 > s3) {
        cout << "s1 > s3 (ASCII comparison)" << endl; // Выполнится
    }

    // Функция compare()
    int result = s1.compare(s2);
    if (result == 0) {
        cout << "Strings are equal" << endl;
    }
}
```



```

} else if (result < 0) {
    cout << "s1 < s2" << endl; // Выполнится
} else {
    cout << "s1 > s2" << endl;
}

// Сравнение подстрок
string text = "Hello World";
string sub = "Hello";
if (text.compare(0, 5, sub) == 0) {
    cout << "First 5 characters equal to 'Hello'" << endl; // Выполнится
}

return 0;
}

```

Поиск и извлечение подстрок

```

#include <iostream>
#include <string>
using namespace std;

int main() {
    string text = "The quick brown fox jumps over the lazy dog";

    // Поиск с начала (find)
    size_t pos = text.find("fox");
    if (pos != string::npos) {
        cout << "'fox' found at position: " << pos << endl; // 16
    }

    // Поиск с конца (rfind)
    pos = text.rfind("the");
    if (pos != string::npos) {
        cout << "Last 'the' found at position: " << pos << endl; // 31
    }

    // Поиск любого символа из набора (find_first_of)
    pos = text.find_first_of("aeiou");
    if (pos != string::npos) {
        cout << "First vowel at position: " << pos << endl; // 2 (e)
    }

    // Извлечение подстроки (substr)
    string sub1 = text.substr(4, 5); // С позиции 4, 5 символов
    cout << "Substring (4,5): " << sub1 << endl; // "quick"

    string sub2 = text.substr(10); // С позиции 10 до конца
    cout << "Substring from 10: " << sub2 << endl; // "brown fox..."

    // Замена подстроки
    text.replace(4, 5, "fast"); // С позиции 4, 5 символов заменяем на "fast"
    cout << "After replace: " << text << endl;

    return 0;
}

```

Модификация строк

```

#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

int main() {
    string text = "Hello World!";

```

```

// Вставка
text.insert(6, "Beautiful ");
cout << "After insert: " << text << endl; // Hello Beautiful World!

// Удаление
text.erase(6, 10); // Удаляем 10 символов с позиции 6
cout << "After erase: " << text << endl; // Hello World!

// Очистка
string temp = "Temporary string";
temp.clear();
cout << "After clear, length: " << temp.length() << endl; // 0

// Изменение размера
text.resize(5);
cout << "After resize(5): " << text << endl; // Hello

text.resize(10, '!');
cout << "After resize(10, '!'): " << text << endl; // Hello!!!!

// Преобразование регистра
string mixed = "HeLIO WoRlD";

// В верхний регистр
transform(mixed.begin(), mixed.end(), mixed.begin(), ::toupper);
cout << "Uppercase: " << mixed << endl; // HELLO WORLD

// В нижний регистр
transform(mixed.begin(), mixed.end(), mixed.begin(), ::tolower);
cout << "Lowercase: " << mixed << endl; // hello world

return 0;
}

```

4. Преобразование между строками и другими типами

std::string ↔ число

```

#include <iostream>
#include <string>
#include <sstream> // Для stringstream
using namespace std;

int main() {
    // 1. Число в строку (C++11 и выше)
    int num = 42;
    double pi = 3.14159;

    string s1 = to_string(num);
    string s2 = to_string(pi);

    cout << "s1: " << s1 << endl; // "42"
    cout << "s2: " << s2 << endl; // "3.141590"

    // 2. Строка в число (C++11 и выше)
    string str_num = "123";
    string str_float = "45.67";

    int n = stoi(str_num);    // string to int
    long l = stol(str_num);   // string to long
    double d = stod(str_float); // string to double

    cout << "n = " << n << endl; // 123
    cout << "d = " << d << endl; // 45.67

    // 3. Универсальный способ (stringstream)

```

```

stringstream ss;

// Число → строка
ss << 100 << " " << 3.14;
string result = ss.str();
cout << "From stringstream: " << result << endl; // "100 3.14"

// Строка → число
ss.clear();
ss.str("200 4.56");
int x;
double y;
ss >> x >> y;
cout << "x = " << x << ", y = " << y << endl; // x = 200, y = 4.56

return 0;
}

```

std::string ↔ C-string

```

#include <iostream>
#include <string>
#include <cstring>
using namespace std;

int main() {
    // string → C-string
    string cpp_str = "Hello C++";
    const char* c_str = cpp_str.c_str();
    const char* c_str2 = cpp_str.data(); // Аналогично c_str()

    cout << "C-string: " << c_str << endl;
    cout << "Length of C-string: " << strlen(c_str) << endl;

    // C-string → string
    char c_style[] = "Hello C";
    string from_c = c_style;
    string from_c2(c_style);

    cout << "From C-string: " << from_c << endl;

    // Копирование в буфер
    char buffer[50];
    cpp_str.copy(buffer, 5, 0); // Копируем 5 символов с позиции 0
    buffer[5] = '\0'; // Не забываем завершающий ноль
    cout << "Copied to buffer: " << buffer << endl; // Hello

    return 0;
}

```

5. Ввод строк

```

#include <iostream>
#include <string>
#include <limits>
using namespace std;

int main() {
    // 1. Оператор >> (до первого пробела)
    string word;
    cout << "Enter a word: ";
    cin >> word;
}

```

```

cout << "You entered: " << word << endl;

// Очистка буфера ввода
cin.ignore(numeric_limits<streamsize>::max(), '\n');

// 2. getline() - чтение всей строки
string line;
cout << "Enter a sentence: ";
getline(cin, line);
cout << "You entered: " << line << endl;

// 3. Чтение до определенного символа
string until_comma;
cout << "Enter text (until comma): ";
getline(cin, until_comma, ',');
cout << "You entered: " << until_comma << endl;

// 4. Чтение нескольких строк
cout << "Enter 3 lines:" << endl;
string lines[3];
for (int i = 0; i < 3; i++) {
    getline(cin, lines[i]);
}

cout << "\nYou entered:" << endl;
for (int i = 0; i < 3; i++) {
    cout << "Line " << i+1 << ": " << lines[i] << endl;
}

return 0;
}

```

6. Практические примеры

Пример 1: Палиндром

```

#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

bool isPalindrome(const string& str) {
    string cleaned;

    // Удаляем пробелы и пунктуацию, приводим к нижнему регистру
    for (char c : str) {
        if (isalnum(c)) {
            cleaned += tolower(c);
        }
    }

    // Проверяем палиндром
    string reversed = cleaned;
    reverse(reversed.begin(), reversed.end());

    return cleaned == reversed;
}

int main() {
    string test1 = "A man, a plan, a canal: Panama";
    string test2 = "Hello World";
    string test3 = "Madam";

    cout << "\"" << test1 << "\" is palindrome? "
         << (isPalindrome(test1) ? "Yes" : "No") << endl;
    cout << "\"" << test2 << "\" is palindrome? "
         << (isPalindrome(test2) ? "Yes" : "No") << endl;
}

```

```
cout << "\"" << test3 << "\" is palindrome? "
    << (isPalindrome(test3) ? "Yes" : "No") << endl;
```

```
    return 0;
}
```

Пример 2: Подсчет слов

```
#include <iostream>
#include <string>
#include <sstream>
using namespace std;
```

```
int countWords(const string& text) {
    stringstream ss(text);
    string word;
    int count = 0;

    while (ss >> word) {
        count++;
    }

    return count;
}
```

```
int main() {
    string text = "C++ is a powerful programming language";

    cout << "Text: " << text << endl;
    cout << "Word count: " << countWords(text) << endl;
```

// Подсчет конкретного слова

```
string search = "programming";
stringstream ss(text);
string word;
int specificCount = 0;
```

```
while (ss >> word) {
    if (word == search) {
        specificCount++;
    }
}
```

```
cout << "Word \"" << search << "\" appears " << specificCount << " times" << endl;
```

```
    return 0;
}
```

Пример 3: Разделение строки на токены

cpp

Copy

Download

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>
using namespace std;
```

```
vector<string> split(const string& str, char delimiter) {
    vector<string> tokens;
    stringstream ss(str);
    string token;
```

```
while (getline(ss, token, delimiter)) {
    if (!token.empty()) {
        tokens.push_back(token);
    }
}
```

```

    }

    return tokens;
}

int main() {
    string csv = "John,Doe,25,New York";
    string path = "/home/user/documents/file.txt";

    cout << "CSV string: " << csv << endl;
    vector<string> csv_parts = split(csv, ',');

    cout << "CSV parts:" << endl;
    for (const string& part : csv_parts) {
        cout << " " << part << endl;
    }

    cout << "\nPath: " << path << endl;
    vector<string> path_parts = split(path, '/');

    cout << "Path parts:" << endl;
    for (const string& part : path_parts) {
        cout << " " << part << endl;
    }

    return 0;
}

```

Пример 4: Форматирование строк

```

#include <iostream>
#include <string>
#include <iomanip>
#include <sstream>
using namespace std;

int main() {
    // Форматирование с помощью stringstream
    stringstream ss;

    int hours = 14;
    int minutes = 5;
    int seconds = 30;

    ss << setfill('0') << setw(2) << hours << ":"
        << setw(2) << minutes << ":" << setw(2) << seconds;

    cout << "Time: " << ss.str() << endl; // 14:05:30

    // Форматирование таблицы
    struct Person {
        string name;
        int age;
        double salary;
    };

    Person people[] = {
        {"John Doe", 30, 50000.50},
        {"Jane Smith", 25, 45000.75},
        {"Bob Johnson", 35, 60000.00}
    };

    cout << "\nEmployee List:" << endl;
    cout << setfill('-') << setw(40) << "" << setfill(' ') << endl;
    cout << left << setw(20) << "Name"
        << setw(10) << "Age"
        << right << setw(10) << "Salary" << endl;
    cout << setfill('-') << setw(40) << "" << setfill(' ') << endl;
}

```

```

for (const auto& person : people) {
    cout << left << setw(20) << person.name
        << setw(10) << person.age
        << right << fixed << setprecision(2) << setw(10) << person.salary
        << endl;
}

return 0;
}

```

7. Работа с файлами и строками

```

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
using namespace std;

int main() {
    // Запись строк в файл
    ofstream outFile("example.txt");
    if (outFile.is_open()) {
        outFile << "Line 1: Hello World\n";
        outFile << "Line 2: C++ Strings\n";
        outFile << "Line 3: File I/O Example\n";
        outFile.close();
        cout << "File written successfully" << endl;
    }

    // Чтение строк из файла
    vector<string> lines;
    ifstream inFile("example.txt");

    if (inFile.is_open()) {
        string line;
        while (getline(inFile, line)) {
            lines.push_back(line);
        }
        inFile.close();

        cout << "\nFile contents:" << endl;
        for (const string& l : lines) {
            cout << l << endl;
        }
    }

    // Поиск в файле
    string searchTerm = "C++";
    inFile.open("example.txt");

    if (inFile.is_open()) {
        string line;
        int lineNumber = 1;
        bool found = false;

        while (getline(inFile, line)) {
            if (line.find(searchTerm) != string::npos) {
                cout << "\nFound \"" << searchTerm << "\" in line "
                    << lineNumber << ": " << line << endl;
                found = true;
            }
            lineNumber++;
        }

        if (!found) {
            cout << "\n\"" << searchTerm << "\" not found" << endl;
        }

        inFile.close();
    }
}

```

```

}

return 0;
}

```

Советы по работе со строками:

1. **Используйте `std::string` вместо C-строк** для безопасности и удобства
2. **Передавайте строки по константной ссылке** в функции: `void func(const string& str)`
3. **Используйте `reserve()`** если знаете примерный размер строки заранее
4. **Избегайте частых перераспределений памяти** - объединяйте операции
5. **Используйте `emplace_back()`** для добавления символов вместо `push_back()`
6. **Для частых поисков** используйте специализированные структуры данных
7. **Для парсинга** используйте `stringstream` или регулярные выражения
8. **Всегда проверяйте результат поиска** на `string::npos`

Список задач

Часть 1.

1. Дана строка, подсчитать сколько раз встречается буква `a`.
2. Выделить символы, заключённые в фигурные скобки.
3. Подсчитать наибольшее число букв `a`, идущих подряд в введенной последовательности символов.
4. Удалить символы, заключённые в фигурные скобки.
5. Дана строка. Подсчитать, сколько раз среди введенных символов встречается буква `b`.
6. Дана строка, заканчивающаяся точкой. Подсчитать, сколько слов в строке.
7. Дана строка, содержащая английский текст. Найти количество слов, начинающихся с буквы `b`.
8. Дана строка. Подсчитать, сколько в ней букв `g`, `k`, `t`.
9. Дана строка. Определить, сколько в ней символов `*` ; :
10. Дана строка, содержащая текст. Найти длину самого короткого слова и самого длинного слова.
11. Дана строка символов, среди которых есть двоеточие. Определить, сколько символов ему предшествует.
12. Дана строка, содержащая текст, заканчивающийся точкой. Вывести на экран слова, содержащие три буквы.
13. Удалить группу символов, расположенных между круглыми скобками включая сами скобки.
14. Если в заданный текст входит каждая из букв слова `key`, тогда напечатать `yes`, иначе `no`.
15. Напечатать заданный текст, удалив из него лишние пробелы, т.е. из нескольких подряд идущих пробелов оставить только один.
16. Логической переменной `b` присвоить значение `true`, если между литерами `'a'` и `'z'` нет иных символов, кроме строчных латинских букв, и значение `false` иначе.
17. Дана строка. Подсчитать сколько раз среди символов встречается символ `<+>` и сколько раз символ `<*>`.
18. Дана строка. Подсчитать общее число вхождений символов `+`, `-` и `*` в строку.
19. Дана строка, заменить в ней все восклицательные знаки точками.
20. Дана строка, заменить в ней каждую точку многоточием.
21. Дана строка. Преобразовать ее, удалив каждый символ `<*>` и повторив каждый символ, отличный от `<*>`.
22. Дана строка, среди символов которой есть двоеточие. Получить все символы, расположенные до первого двоеточия включительно.
23. Дана строка, среди символов которой есть двоеточие. Получить все символы, расположенные после первого двоеточия включительно.
24. Дана строка, среди символов которой есть двоеточие. Получить все символы, расположенные между первым и вторым двоеточиями. Если второго двоеточия нет, то получить все символы после первого двоеточия.
25. Дана строка. Подсчитать наибольшее количество идущих подряд пробелов.

Часть 2.

Дана строка. Группы символов, разделённых пробелами (одним или несколькими) и не содержащим пробелов внутри себя будем называть словами. Задания (1-9):

1. подсчитать количество букв `"a"` в последнем слове данной последовательности.
2. найти количество слов, начинающихся с буквы `"c"`.
3. найти количество слов, у которых первый и последний символы совпадают.

4. подсчитать количество слов в данной последовательности.
5. преобразовать данную последовательность, заменяя всякое вхождение слова "это" на слово "то".
6. найти длину самого короткого слова.
7. найти длину самого длинного слова.
8. удалить все символы, не являющиеся буквами.
9. заменить все малые буквы одноимёнными большими.
10. Дана строка. Определить, сколько раз входит в нее группа букв abc.
11. Дана строка. Подсчитать, сколько различных символов встречается в ней. Вывести их на экран.
12. Дана строка. Подсчитать самую длинную последовательность подряд идущих букв а.
13. Найти последнее самое короткое слово предложения.
14. Имеется строка, содержащая буквы латинского алфавита и цифры. Вывести на экран длину наибольшей последовательности цифр, идущих подряд.
15. Дан набор слов, разделенных точкой с запятой (;). Набор заканчивается двоеточием (:). Определить, сколько в нем слов, заканчивающихся буквой а.
16. Найти первое самое короткое слово предложения.
17. Дана строка. Указать те слова, которые содержат хотя бы одну букву к.
18. В строке заменить все двоеточия (:) точкой с запятой (;). Подсчитать количество замен.
19. В строке удалить символ «двоеточие» (:) и подсчитать количество удаленных символов.
20. В строке между словами вставить вместо пробела запятую и пробел.
21. Составить процедуру, заменяющую в исходной строке символов все единицы нулями и все нули единицами. Замена должна выполняться, начиная с заданной позиции строки.
22. Найти самое длинное слово в предложении.
23. Найти первое симметричное слово в предложении.
24. Заменить заданное слово предложения на другое слово